



معماری کامپیوتر

بهار ۱۴۰۵

استاد: دکتر سربازی آزاد

محسن صلاح - ۴۰۳۱۰۶۲۳۸

تمرین عملی اول

۱ مقدمه

در این تمرین، یک واحد حساب و منطق (ALU) ترکیبی ۳۲ بیتی را طراحی کردیم که می‌تواند عملیات جمع، تفریق و ضرب را انجام دهد. مطابق صورت تمرین، ورودی‌ها و خروجی‌های این واحد به این صورت هستند:

• ورودی:

۱. A: عملوند اول (۳۲ بیت)
۲. B: عملوند دوم (۳۲ بیت)
۳. ALU_Ctrl: سیگنال کنترلی تعیین نوع علامت (۳ بیت)

• خروجی:

۱. Result_Lo: ۳۲ بیت کم‌ارزش حاصل عملیات (۳۲ بیت)
۲. Result_Hi: ۳۲ بیت پرارزش حاصل عملیات (تنها در عملیات ضرب کاربرد دارد) (۳۲ بیت)
۳. C (Carry): پرچم رقم نقلی (۱ بیت)
۴. V (Overflow): پرچم سرریز (۱ بیت)
۵. Z (Zero): پرچم صفر (۱ بیت)
۶. N (Negative): پرچم منفی (۱ بیت)

و خروجی‌ها نیز به ازای عملیات مختلف به این صورت هستند:

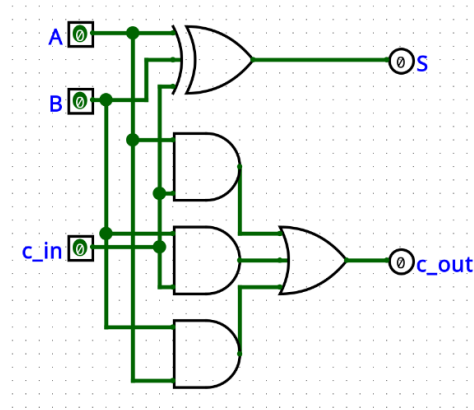
جدول ۱: کدهای کنترلی واحد حساب و منطق

کد کنترلی	نام دستور	شرح عملیات	پرچم‌های فعال
۰۰۰	ADD_U	جمع دو عدد بدون علامت ($A + B$)	C, Z, V
۰۰۱	ADD_S	جمع دو عدد علامت‌دار ($A + B$)	V, N, Z
۰۱۰	SUB_U	تفریق دو عدد بدون علامت ($A - B$)	C, Z, V
۰۱۱	SUB_S	تفریق دو عدد علامت‌دار ($A - B$)	V, N, Z
۱۰۰	MUL_U	ضرب ترکیبی دو عدد بدون علامت ($A \times B$)	Z
سایر حالت‌ها	NOP	خروجی‌ها صفر باشند.	هیچ‌کدام

۲ عملیات جمع و تفریق

جمع‌کننده به صورت انتشار رقم نقلی (Carry Ripple Adder) و تفریق‌کننده نیز به صورت جمع ممتهم دو پیاده‌سازی شد.

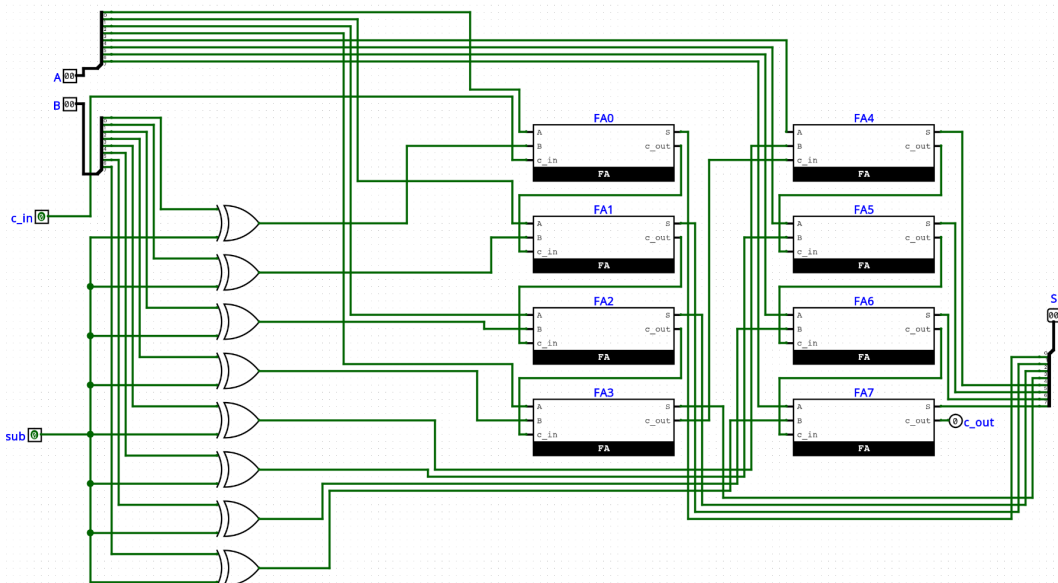
ابتدا، یک جمع‌کننده کامل (Full Adder یا به اختصار FA) ساختیم:



شکل ۱: جمع‌کننده کامل (FA)

سپس با کنار هم قرار دادن ۸ واحد FA، یک جمع‌کننده/تفریق‌کننده ۸ بیتی ساختیم: با استفاده از ابزار Splitter در Logisim، بیت‌های دو ورودی ۸ بیتی A و B را جدا کردیم. بیت‌های A مستقیماً به ورودی اول FA ها وصل شدند. بیت‌های B نیز با ورودی تک‌بیتی sub، XOR شده و سپس به ورودی دیگر جمع‌کننده‌ها رفتند. همچنین ورودی تک‌بیتی c_in به ورودی c_in جمع‌کننده اول وصل شد. سپس، c_out هر جمع‌کننده را به ورودی c_in جمع‌کننده بعدی وصل کردیم.

با استفاده از Splitter، خروجی S جمع‌کننده‌ها را کنار هم قرار دادیم تا یک خروجی ۸ بیتی ساخته شود. c_out جمع‌کننده آخر نیز همان خروجی c_out مدار است:

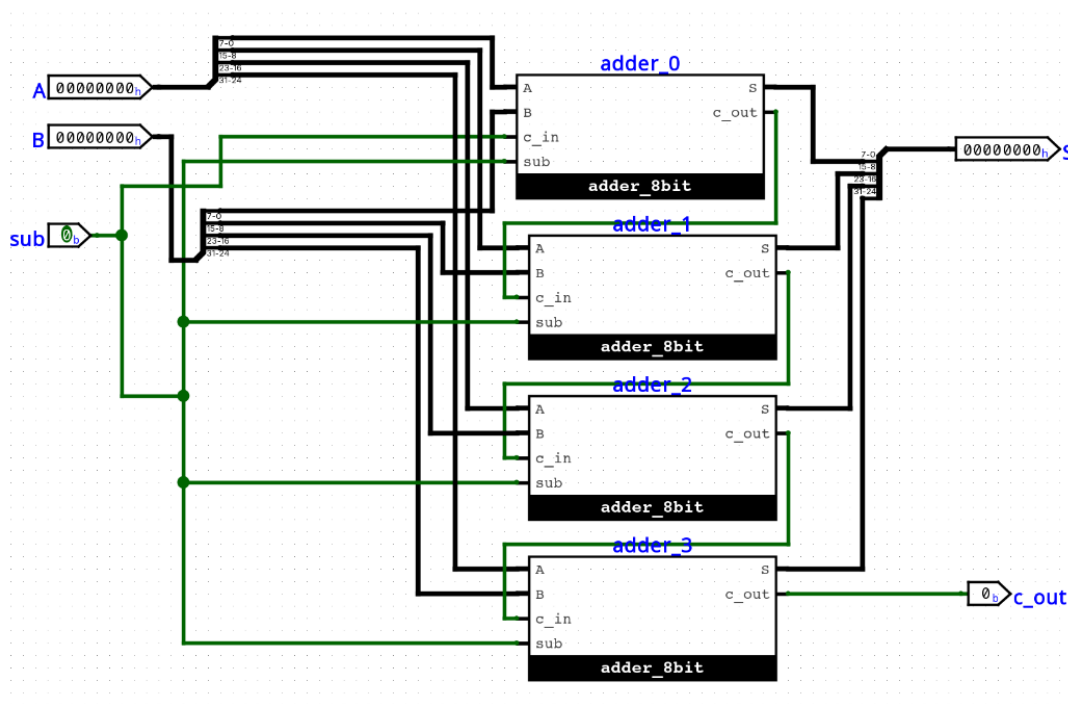


شکل ۲: جمع/تفریق‌کننده ۸ بیتی

با XOR کردن ورودی sub با بیت‌های B، عملیات تفریق با استفاده از مکمل دو (Two's Complement) انجام می‌شود؛ با XOR کردن، مکمل یک B ساخته می‌شود ($\overline{B}$)، سپس همان طور که در بخش بعدی می‌بینیم، چون sub را به ورودی c_in جمع/تفریق کننده‌ی اول نیز وصل کردیم، ورودی‌ها با ۱ جمع شده و فرمول زیر اجرا می‌شود:

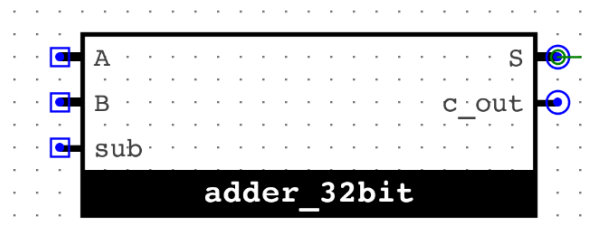
$$A - B = A + (-B) = A + \overline{B} + 1$$

برای ساخت جمع/تفریق کننده‌ی ۳۲ بیتی، به طور مشابه ۴ واحد از جمع/تفریق کننده‌های ۸ بیتی را کنار هم قرار دادیم. بیت‌های دو ورودی ۳۲ بیتی A و B را به ۴ بخش ۸ بیتی تبدیل کرده و به هر یک از چهار پیمانه (module) دادیم. ورودی sub نیز به c_in پیمانه‌ی اول و sub و هر چهار پیمانه متصل شد. خروجی‌ها نیز مشابه بخش قبلی هستند:



شکل ۳: جمع/تفریق کننده ۳۲ بیتی

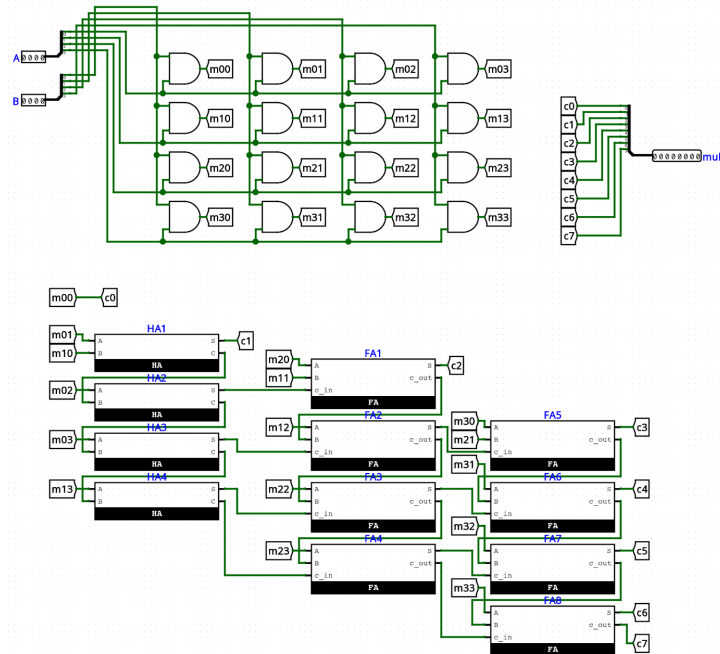
در نهایت، واحد جمع/تفریق کننده (که آن را adder_32bit می‌نامیم) به این صورت است:



شکل ۴: پیمانه‌ی adder_32bit

۳ عملیات ضرب

ابتدا یک ضرب‌کننده ۴ بیتی ترکیبی را به این صورت ساختیم: (در این مرحله علاوه بر FA که در بخش قبل ساخته بودیم، از Half Adder (HA) هم استفاده کردیم)

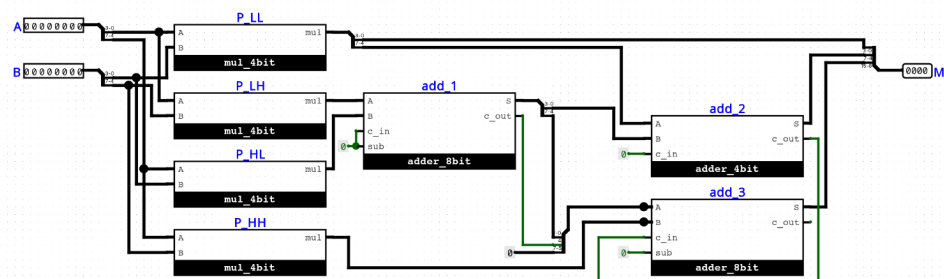


شکل ۵: ضرب‌کننده ۴ بیتی

سپس، سعی کردیم با قرار دادن چهار واحد از ضرب‌کننده‌های ۴ بیتی کنار هم و با استفاده از الگوریتم تقسیم و غلبه (Divide and Conquer)، ضرب‌کننده ۸ بیتی بسازیم. از نظر ریاضی، اگر دو عدد ۸ بیتی A و B را به دو بخش ۴ بیتی کم‌ارزش (Low) و پرارزش (High) تقسیم کنیم، عملیات ضرب به شکل زیر بسط داده می‌شود:

$$\begin{aligned} A &= A_H \cdot 2^4 + A_L \\ B &= B_H \cdot 2^4 + B_L \\ A \times B &= (A_H \cdot 2^4 + A_L) \times (B_H \cdot 2^4 + B_L) \\ &= (A_H \times B_H)2^8 + (A_H \times B_L + A_L \times B_H)2^4 + (A_L \times B_L) \end{aligned}$$

بر اساس این منطق، مدار ضرب‌کننده ۸ بیتی به شکل زیر پیاده‌سازی شده است:



شکل ۶: ضرب‌کننده ۸ بیتی

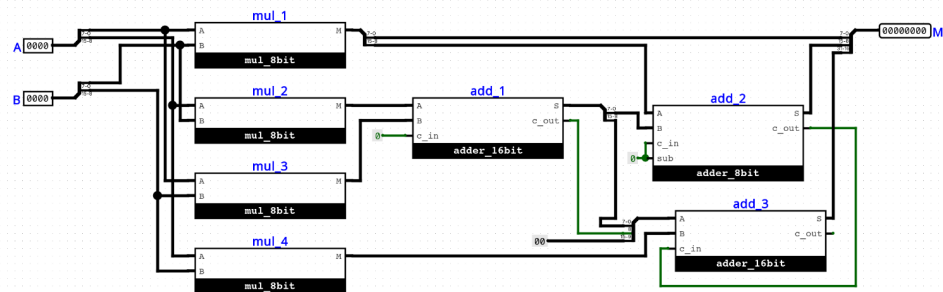
همان‌طور که در تصویر مشخص است، روند اجرای مدار شامل مراحل زیر است:

- **جداسازی ورودی‌ها:** ابتدا با استفاده از مازول Splitter، ورودی‌های ۸ بیتی A و B هر کدام به دو بخش ۴ بیتی تقسیم می‌شوند.
- **حاصل ضرب‌های جزئی:** این بخش‌ها به چهار واحد ضرب‌کننده‌ی ۴ بیتی وارد می‌شوند تا چهار حاصل ضرب جزئی (P_{LL} , P_{LH} , P_{HL} , P_{HH}) تولید شود.
- **جمع و شیف:**

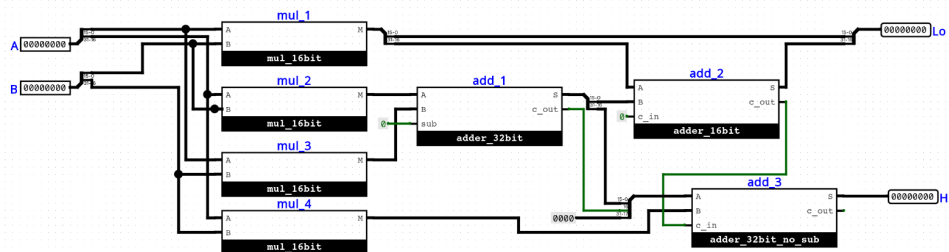
- با استفاده از یک جمع‌کننده‌ی ۸ بیتی (add_1)، عبارات میانی ($P_{LH} + P_{HL}$) با هم جمع می‌شوند.
- ۴ بیت کم‌ارزش خروجی P_{LL} مستقیماً به عنوان ۴ بیت اول خروجی نهایی استفاده می‌شود. ۴ بیت پرارزش آن به کمک یک جمع‌کننده‌ی ۴ بیتی (add_2) با ۴ بیت کم‌ارزش حاصل جمع میانی (add_1) جمع شده و ۴ بیت دوم خروجی نهایی را می‌سازد.
- در نهایت، یک جمع‌کننده‌ی ۸ بیتی دیگر (add_3)، حاصل ضرب پرارزش (P_{HH}) را با بخش‌های باقی‌مانده از جمع‌های قبلی (همراه با لحاظ کردن رقم نقلی یا c_out از add_2 به عنوان c_in و همچنین c_out بلوک add_1) جمع می‌کند تا ۸ بیت پرارزش نهایی تولید شود.

- **الحاق خروجی‌ها:** در انتها، با استفاده مجدد از یک Splitter (به صورت معکوس)، بخش‌های محاسبه‌شده با یکدیگر ترکیب (الحاق) می‌شوند تا خروجی نهایی ۱۶ بیتی (M) ساخته شود.

سپس به طریق کاملاً مشابه، از این پیمان استفاده کردیم تا ضرب‌کننده‌ی ۱۶ بیتی بسازیم و به کمک آنها هم ضرب‌کننده‌ی ۳۲ بیتی ساخته شد:

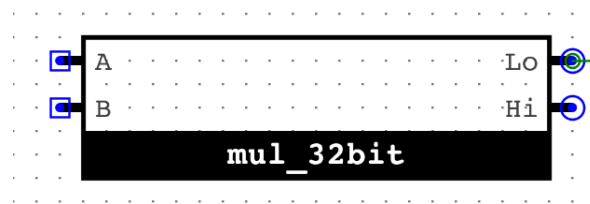


شکل ۷: ضرب‌کننده‌ی ترکیبی ۱۶ بیتی



شکل ۸: ضرب‌کننده‌ی ترکیبی ۳۲ بیتی

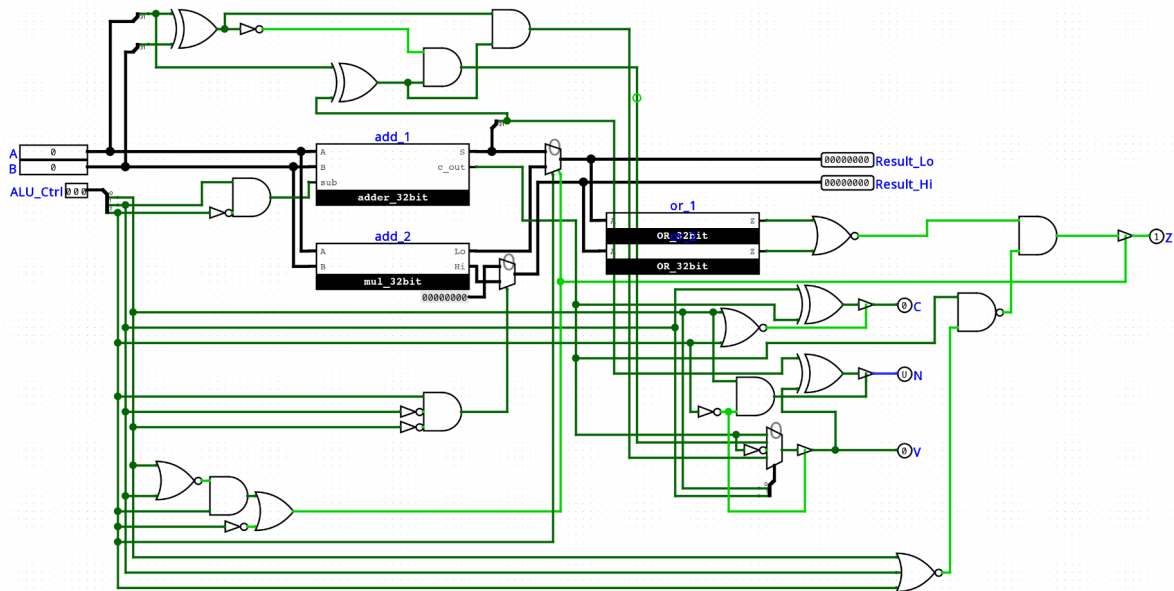
در نهایت، واحد ضرب‌کننده (که آن را mul_32bit می‌نامیم) به این صورت است:



شکل ۹: پیمانه‌ی mul_32bit

۴ واحد حساب و منطق (ALU)

مدار طراحی شده‌ی این قسمت به این صورت است:



شکل ۱۰: واحد حساب و منطق ۳۲ بیتی

در ادامه، هر بخش را جداگانه توضیح می‌دهیم.

۱.۴ تولید خروجی‌ها (مسیر داده)

ورودی‌های ۳۲ بیتی A و B را هم‌زمان به ورودی‌های پیمانه‌ی `adder_32bit` و `mul_32bit` می‌دهیم تا هر دو عملیات به موازات هم انجام شوند (تفاوت بین حالت‌های جمع و تفریق در بخش بعد توضیح داده می‌شود). دو مالتی‌پلکسر (MUX) دو به یک در مدار قرار داده (bitwidth هر دو ۳۲ بیت است) و خروجی‌های آن‌ها را به ترتیب `Result_Lo` و `Result_Hi` می‌نامیم. خروجی اصلی واحد جمع‌کننده و خروجی `Lo` ضرب‌کننده به مالتی‌پلکسر اول متصل شده‌اند تا `Result_Lo` را تشکیل دهند. خروجی `Hi` ضرب‌کننده نیز به ورودی یک مالتی‌پلکسر دوم وصل شده و ورودی دیگر آن به مقدار ثابت صفر وصل شده تا `Result_Hi` را کنترل کند.

۲.۴ رمزگشایی کد کنترل

سیگنال کنترلی `ALU_Ctrl` (که آن را به اختصار `AC` می‌نامیم) سه بیت دارد: بیت پرارزش (`AC۲`) نوع عملیات پایه (جمع/تفریق یا ضرب) را مشخص می‌کند؛ بیت میانی (`AC۱`) جمع یا تفریق بودن را تعیین می‌کند؛ و بیت کم‌ارزش (`AC۰`) نیز بی‌علامت یا علامت‌دار بودن عملیات را مشخص می‌سازد.

در حالت $AC_2 = 0$ ، باید خروجی مدار جمع/تفریق‌کننده در `Result_Lo` نوشته شود، بنابراین این بیت را به خط انتخاب (select) مالتی‌پلکسر اول وصل می‌کنیم. همچنین خروجی `Result_Hi` تنها در حالت $AC = 100$ فعال بوده و در حالت‌های دیگر صفر است. بنابراین به کمک یک گیت AND و گیت‌های NOT، مقدار $AC_2 AC_1 AC_0$ را ساخته و به خط انتخاب مالتی‌پلکسر دیگر وصل می‌کنیم.

برای کنترل عملگر جمع یا تفریق، سیگنال $\overline{AC_2} AC_1$ تولید شده و به ورودی `sub` در پیمانه‌ی جمع‌کننده متصل شده است تا در صورت صدور دستورات تفریق ($AC_2 AC_1 = 01$)، مدار به صورت مکمل دو عمل کند.

۳.۴ تولید پرچم‌ها (Flags)

خروجی هر یک از پرچم‌ها توسط یک بافر سه‌حالت (Tri-state Buffer) کنترل می‌شود تا پرچم‌ها تنها در دستوراتی که از نظر منطقی معتبر هستند خروجی دهند و در سایر حالت‌ها در وضعیت امپدانس بالا (High Impedance) قرار گیرند. معادلات فعال‌ساز (Enable) و منطق محاسبه‌ی هر پرچم بر اساس مدار به شرح زیر است:

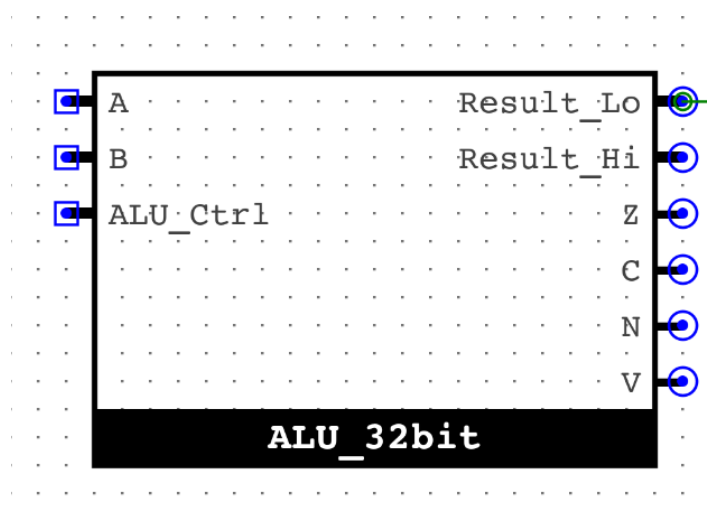
$$C = AC_1 \oplus c_out$$

$$V = (\overline{AC_1 AC_0})(c_out) + (\overline{AC_1 AC_0})((A_{31} \odot B_{31}) \cdot (A_{31} \oplus S_{31})) \\ + (AC_1 \overline{AC_0})(\overline{c_out}) + (AC_1 AC_0)((A_{31} \oplus B_{31}) \cdot (A_{31} \oplus S_{31}))$$

$$Z = (\text{OR}(\text{Result_Lo}) \downarrow \text{OR}(\text{Result_Hi})) \cdot ((c_out) \uparrow (AC_2 \downarrow AC_1 \downarrow AC_0))$$

$$N = S_{31} \oplus V$$

در نهایت، واحد محاسبه و منطق (که آن را ALU_32bit می‌نامیم) به این صورت است:



شکل ۱۱: پیمانه‌ی ALU_32bit

۵ اجرای آزمون

به کمک اسکریپت‌های داده شده، مدار را سنتز کرده و تست‌ها را اجرا می‌کنیم. نتیجه‌ی آزمون به این صورت است که موفقیت‌آمیز بودن تمامی عملیات‌ها را تایید می‌کند:

```
1 mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$ ./validate.sh ./HW1/tb.v ~/
  ↳ logisim_evolution_workspace/HW1_403106238.circ.tmp/HW1_403106238.circ.
  ↳ tmp
2 ./HW1/tb.v:34: Warning: Calling system function $random() as a task.
3 ./HW1/tb.v:34: The functions return value will be ignored.
4 ADD_U
5 SUB_U
6 ADD_S
7 SUB_S
8 MUL_U
9 ACCEPTED
10      50 / 50
11 mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$
```
